AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI

Generative AI Micro Project (7CS1018)

Submitted by

Chalini M – 2023BCSE07AED440

Spandana Basava Kulal – 2023BCSE07AED441

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (DA)

Under the Supervision of

Dr. Marimganti Srinivas



ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU NOVEMBER - 2026

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

DEPARTMENT OF CSE (AI/ML)

CERTIFICATE

This is to certify that the In-House Project work entitled **"AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI"** is the Bonafide work done by Chalini M (2023BCSE07AED440), Spandana Basava Kulal (2023BCSE07AED441) submitted in partial fulfilment of the requirements for the award of the degree Bachelor of Technology in **Computer Science and Engineering** during the year 2026.

Supervisor
Associate Professor
Dept. of CSE (Specialization)

ALLIANCE UNIVERSITY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

This is to declare that the report titled **"AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI"** has been made for the partial fulfilment of the Programme - Bachelor of Technology in **Computer Science and Engineering**, under the guidance of **Dr. Marimganti Srinivas**. We confirm that this report truly represents my work undertaken as a part of **our Gen AI Micro Project** work. This work is not a replication of work done previously by any other person. We also confirm that the contents of the report and the views contained

therein have been discussed and deliberated with the Mentor.

NAME	REGESTRATION NO.	SIGNATURE
Chalini M	2023BCSE07AED440	
Spandana Basava Kulal	2023BCSE07AED441	

ALLIANCE UNIVERSITY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people that made it possible, whose constant guidance and encouragement crown all the efforts successfully.

We are very thankful to my supervisor **Dr. Marimganti Srinivas, Associate Professor** for his sustained inspiring guidance and cooperation throughout the process of this project. Their wise counsel and valuable suggestions are invaluable.

We would like to thank **Dr. K Rama Lakshmi,** Head of the Department and **Dr. V Vivek**, Deputy Director, for their encouragement and cooperation at various levels of Project.

We avail this chance to express my deep sense of gratitude and hearty thanks to the Management of Alliance University, for providing world class infrastructure, congenial atmosphere, and encouragement.

We express my deep sense of gratitude and because of the teaching and non-teaching staff at our department who stood with me during the project and helped me to make it a successful venture.

We place highest regards to my parents, my friends and well-wishers who helped plenty in making the report of this project.

Chalini M

Spandana Basava Kulal

ABSTRACT

The increasing competition in the job market has made resume optimization and interview preparation essential for job seekers. However, candidates often face difficulties in identifying the skills required for a particular job, aligning their resumes with job descriptions, and preparing for interviews effectively. Traditional approaches to career preparation are time-consuming and generally provide limited personalization.

This project presents an **AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI**, an intelligent application designed to support candidates throughout the job preparation process. The system uses Generative AI and Natural Language Processing techniques to analyze resumes and compare them with target job descriptions. It employs semantic similarity using Nomic Embeddings to identify the degree of alignment between candidate profiles and job requirements. The application also generates personalized resume improvement suggestions, identifies matching and unclear skills, and provides role-specific technical, project-based, and behavioral interview questions.

Furthermore, the system incorporates an adaptive interview chatbot powered by the Llama 3.2 language model through Ollama. It conducts interactive practice interviews by generating follow-up questions based on the candidate's responses. After the interview, the system produces an AI-generated evaluation report highlighting performance, strengths, areas for improvement, and a personalized learning plan. The application is developed using Python and Streamlit, with PyMuPDF for PDF-based resume text extraction.

The proposed system integrates resume analysis, resume improvement, interview preparation, adaptive interview practice, and performance feedback into a unified platform. It aims to make career preparation more personalized, accessible, and efficient while helping candidates identify skill gaps and improve their interview readiness.

LIST OF SYMBOLS & ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CV	Curriculum Vitae
JD	Job Description
LLM	Large Language Model
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
UI	User Interface
UX	User Experience
TF-IDF	Term Frequency-Inverse Document Frequency
FAISS	Facebook AI Similarity Search
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
CPU	Central Processing Unit
RAM	Random Access Memory

CONTENTS

	Page No
Title Page	**1**
Certificate	**2**
Declaration	**3**
Acknowledgement	**4**
Abstract	**5**
List of Symbols & Abbreviations	**6**
Contents	**7**
1. Introduction	**8**
2. Problem Statement, Objectives and Scope	**9-11**
2.1 Problem Statement	
2.2 Objectives	
2.3 Scope of the Project	
3. Literature Review / Existing Systems	**12**
4. Proposed Methodology and System Architecture	**13-15**
4.1 Overall Workflow	
4.2 Main Components	
4.3 Processing and Generation Procedure	
5. Tools, Technologies and Knowledge Sources	**16**
5.1 Knowledge Sources / Dataset	
5.2 Reproducibility Details	
6. Implementation	**17**
6.1 Data Preparation	
6.2 Embedding and Similarity Analysis	
6.3 Resume and Job Description Analysis	
6.4 LLM Integration	
6.5 User Interface	
6.6 Error Handling	
7. Experiments, Results and Discussion	**18-20**
7.1 Evaluation Design	
7.2 Results and Discussion	
8. Limitations, Ethical Considerations and Risk Mitigation	**21-22**
8.1 Limitations	
8.2 Ethical Considerations	
8.3 Risk Mitigation Strategies	
9. Conclusion and Future Enhancements	**23**
9.1 Conclusion	
9.2 Future Enhancements	
References	**24**

1. INTRODUCTION

In today's competitive job market, securing employment requires more than academic qualifications. Candidates must present their skills effectively through well-structured resumes and demonstrate their technical knowledge, communication abilities, analytical thinking, and problem-solving skills during interviews. Organizations expect candidates to possess relevant technical expertise, practical experience, adaptability, and the ability to apply their knowledge to real-world situations. As a result, preparing for recruitment processes has become an important challenge for students, fresh graduates, and experienced professionals.

A resume plays a significant role in the recruitment process because it provides employers with an overview of a candidate's educational qualifications, technical skills, projects, internships, certifications, professional experience, and achievements. A well-structured resume helps recruiters understand whether a candidate's profile matches the requirements of a particular job role. However, many candidates face difficulties in preparing resumes that effectively highlight their qualifications and align with specific job descriptions. They may include unnecessary information, fail to highlight important technical skills, or describe their projects without explaining their actual contributions and achievements.

Another major challenge is identifying the skills required for a particular job position. Different organizations specify different technical requirements, programming languages, frameworks, tools, and experience levels. Candidates must carefully examine job descriptions and compare them with their existing qualifications to understand their suitability for a position. Manually performing this comparison can be time-consuming and may result in overlooking important requirements. Candidates may also struggle to identify the difference between skills they already possess, skills that are not clearly represented in their resumes, and skills that require further development.

Traditional resume preparation involves manually reviewing job descriptions, identifying relevant skills, modifying resume content, and preparing possible interview questions. This process is time-consuming and may not always provide sufficient guidance about skill gaps or areas requiring improvement. Similarly, conventional interview preparation methods often depend on generic question banks, online tutorials, and predefined question

lists. These resources may not consider an individual's resume, academic projects, technical background, or target job role, resulting in less personalized preparation.

Interview preparation is equally important because candidates are evaluated on their technical knowledge, practical implementation skills, communication abilities, confidence, and problem-solving approach. Technical interviews may involve questions related to programming languages, data structures, algorithms, databases, artificial intelligence, machine learning, and other domain-specific concepts. Project-based interviews require candidates to explain their project objectives, methodologies, technologies, implementation procedures, and challenges. Behavioral interviews focus on teamwork, leadership, adaptability, decision-making, and communication skills. Therefore, candidates require a structured preparation approach that covers different interview categories.

Recent advancements in Artificial Intelligence (AI), Natural Language Processing (NLP), and Generative Artificial Intelligence have created new opportunities for improving career preparation systems. NLP enables computers to process and understand human language, making it possible to extract meaningful information from resumes and job descriptions. Semantic similarity techniques help identify relationships between two text documents based on their contextual meaning rather than relying only on exact keyword matches. These techniques can be used to compare candidate profiles with job requirements and estimate their relevance.

Generative AI further enhances these capabilities through Large Language Models (LLMs), which can understand textual context and generate human-like responses. These models can assist in resume summarization, skill identification, resume improvement, interview question generation, and personalized feedback. They can also support interactive conversations by generating follow-up questions based on previous responses, creating a more realistic interview preparation experience.

To address these challenges, the **AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI** is developed as an intelligent application that integrates resume analysis, job description matching, resume improvement, interview preparation, adaptive mock interviews, and performance evaluation into a single platform. The system uses Natural Language Processing, semantic similarity techniques, and Large

Language Models to analyze candidate resumes and compare them with the requirements of specific job roles.

The application initially accepts a candidate's resume in PDF format and extracts its textual information using the PyMuPDF library. The candidate can also enter a target job role and provide the corresponding job description. The extracted resume content and job description are converted into numerical representations known as semantic embeddings using the Nomic Embedding model. These embeddings capture contextual information from the input text. Cosine similarity is then used to estimate the semantic relationship between the resume and job description.

Based on this analysis, the system identifies matching skills, missing or insufficiently represented requirements, and areas where the resume can be improved. The Llama 3.2 Large Language Model, executed locally through Ollama, is used to analyze the candidate's information and generate personalized recommendations. The resume improvement module provides suggestions for enhancing resume bullet points, improving clarity, highlighting relevant technical skills, and presenting project contributions more effectively without introducing unsupported qualifications or experience.

In addition to resume analysis, the application provides a personalized interview preparation module that generates technical, project-based, and behavioral interview questions according to the candidate's resume and selected job role. This allows candidates to prepare for interviews using questions related to their actual technical background, projects, and career objectives rather than depending entirely on generic question banks.

An important feature of the proposed application is its adaptive mock interview chatbot. It uses Llama 3.2 through Ollama to conduct interactive interview sessions. The chatbot presents questions to the candidate and generates relevant follow-up questions based on the responses provided during the conversation. This helps simulate an interview environment and allows candidates to practice explaining their knowledge, technical decisions, and project experiences.

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

3.1 Problem Statement

In the current competitive employment environment, candidates often face difficulties in preparing resumes that align with specific job descriptions and identifying the skills required for their desired positions. Manually comparing resumes with job requirements, identifying skill gaps, improving resume content, and preparing for interviews can be time-consuming and challenging.

Existing interview preparation resources frequently provide generic questions that do not consider an individual's technical background, projects, skills, or target job role. Additionally, candidates may lack an effective mechanism to evaluate their interview responses and identify areas that require further improvement.

Therefore, there is a need for an integrated AI-based system that can analyze resumes against job descriptions, identify matching and unclear skills, suggest relevant resume improvements, generate personalized interview questions, conduct adaptive practice interviews, and provide performance feedback to support continuous career preparation.

3.2 Objectives

The primary objective of this project is to develop an AI-powered resume and interview assistant that provides personalized career preparation using Generative AI and Natural Language Processing techniques.

The specific objectives are:

1. **Resume Text Extraction:** To extract relevant information from PDF resumes using PyMuPDF for further analysis.
2. **Resume and Job Description Analysis:** To compare candidate resumes with target job descriptions using semantic similarity techniques and identify their alignment.
3. **Skill Matching and Gap Identification:** To identify matching skills, missing or unclear requirements, and weakly represented skills based on resume evidence.
4. **Resume Improvement:** To generate role-specific suggestions and improve existing resume bullet points while maintaining the accuracy of candidate information.

5. **Personalized Interview Preparation:** To generate technical, project-based, and behavioral interview questions according to the candidate's resume and target job role.
6. **Adaptive Interview Practice:** To develop an interactive chatbot that generates relevant follow-up questions based on the candidate's previous responses.
7. **Interview Performance Evaluation:** To evaluate submitted interview responses and generate feedback highlighting strengths and areas for improvement.
8. **Personalized Learning Plan:** To recommend relevant topics, practice questions, and learning activities based on the interview evaluation.

3.3 Scope of the Project

The scope of the proposed system covers the integration of AI-based resume analysis and personalized interview preparation into a single application.

The major functionalities included within the project scope are:

1. **Resume Upload and Processing:** Supports uploading PDF resumes and extracting selectable text for analysis.
2. **Job Description Input:** Allows candidates to enter their target job role and paste the corresponding job description.
3. **Semantic Similarity Analysis:** Uses Nomic Embeddings to calculate semantic similarity between resume content and job description requirements.
4. **Resume Analysis:** Generates a structured report containing resume summaries, job requirements, matching skills, unclear requirements, and overall assessment.
5. **Resume Enhancement Suggestions:** Provides recommendations for improving resume content, formatting, clarity, and role-specific relevance.
6. **Interview Question Generation:** Generates personalized technical, project-based, and behavioral questions.
7. **Interactive Interview Chatbot:** Conducts practice interviews using the Llama 3.2 language model through Ollama and generates adaptive follow-up questions.
8. **Feedback and Evaluation:** Produces AI-generated interview feedback based on submitted answers, including approximate performance assessments and improvement areas.
9. **Learning Recommendations:** Provides a personalized learning plan to help candidates improve their knowledge and interview performance.

10. **Report Download:** Allows users to download generated resume improvement, interview preparation, and evaluation reports.

The application is developed using Python and Streamlit, with Ollama for local language model processing, Nomic Embeddings for semantic similarity, and PyMuPDF for PDF text extraction.

3. LITERATURE REVIEW / EXISTING SYSTEMS

1. Automated Resume Parsing: A Natural Language Processing Approach (2023)

- Conference: IEEE 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS).
- Algorithm Used: Natural Language Processing (NLP), Named Entity Recognition (NER), Regular Expressions.
- Key Methodology: The research focuses on extracting important information such as skills, educational qualifications and work experience from resumes using NLP techniques.
- Limitation: The approach primarily concentrates on information extraction rather than comprehensive candidate evaluation.
- Research Gap: Semantic understanding and personalized career recommendations can be further incorporated.

2. An Advanced Real-Time Job Recommendation System and Resume Analyser (2023)

- Conference: IEEE International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS).
- Algorithm Used: Graph Neural Networks, GloVe, TF-IDF, Cosine Similarity.
- Key Methodology: The system analyzes candidate resumes and compares extracted information with job requirements to recommend suitable employment opportunities.
- Limitation: The matching process depends on extracted features and available job information.
- Research Gap: Context-aware resume analysis and personalized interview preparation can be integrated.

3. ResumAI: Revolutionizing Automated Resume Analysis and Recommendation with Multi-Model Intelligence (2023)

- Conference: IEEE Global Conference on Information Technologies and Communications (GCITC).

- Algorithm Used: BERT, TF-IDF, Naive Bayes, Linear SVC and KNN.
- Key Methodology: Multiple machine-learning models are combined to analyze resume information and generate recommendations.
- Limitation: The system mainly concentrates on resume classification and recommendations.
- Research Gap: LLM-based explanations and interactive interview assistance can improve the overall system.

4. Semantic Similarity Analysis for Resume Filtering using PySpark (2024)

- Conference: IEEE 9th International Conference for Convergence in Technology (I2CT).
- Algorithm Used: NLP, Cosine Similarity and PySpark.
- Key Methodology: Semantic similarity techniques are applied to compare resumes with job descriptions and identify suitable candidates.
- Limitation: The approach does not provide comprehensive resume improvement suggestions.
- Research Gap: Personalized skill-gap identification and improvement recommendations are required.

5. Development of a Recommendation System for Resume Tracking and Job Suggestions (2024)

- Conference: IEEE Delhi Section Flagship Conference (DELCON).
- Algorithm Used: Generative AI, NLP and Web Scraping.
- Key Methodology: The system analyzes resume information and recommends relevant job opportunities using AI-based techniques.
- Limitation: The primary focus is on job recommendations and resume tracking.
- Research Gap: Personalized interview questions and candidate preparation guidance can be incorporated.

6. Optimizing Job Matching Through Automated Resume Screening with NLP and Machine Learning (2024)

- Conference: IEEE International Conference on Intelligent Signal Processing and Effective Communication Technologies (INSPECT).
- Algorithm Used: TF-IDF, SVM, Logistic Regression, Random Forest and KNN.
- Key Methodology: NLP preprocessing and machine-learning classification techniques are applied to automate resume screening and job matching.
- Limitation: Traditional machine-learning techniques may not fully capture contextual relationships between skills.
- Research Gap: Semantic embeddings and LLMs can improve contextual matching and personalized feedback.

7. Resume Ranking with TF-IDF, Cosine Similarity and Named Entity Recognition (2024)

- Conference: IEEE First International Conference on Data, Computation and Communication (ICDCC).
- Algorithm Used: TF-IDF, Cosine Similarity and Named Entity Recognition.
- Key Methodology: Relevant resume information is extracted and candidates are ranked based on similarity with job requirements.
- Limitation: Keyword-based ranking may overlook equivalent skills expressed using different terminology.
- Research Gap: Contextual semantic matching and explainable candidate evaluation are needed.

8. AI-Based Multimodal Resume Ranking Web Application for Large Scale Job Recruitment (2024)

- Conference: IEEE International Conference on Data Analytics and Processing (IDAP).
- Algorithm Used: YOLOv9, EasyOCR, mBERT and GLiNER.
- Key Methodology: Visual processing, optical character recognition and language models are combined to extract and evaluate resume information.

- Limitation: Multimodal processing can increase computational complexity.
- Research Gap: Lightweight systems can provide accessible resume analysis and personalized career preparation.

9. LLM-Driven Resume Parsing and Job Description Matching: Evaluation on 998 Real Resumes (2026)

- Conference: IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC).
- Algorithm Used: Large Language Models, RAG, Fine-tuning and Prompt Engineering.
- Key Methodology: LLM-based approaches are evaluated for resume parsing and job-description matching across different occupational areas.
- Limitation: Performance depends on model quality, prompting techniques and evaluation conditions.
- Research Gap: Resume improvement, adaptive mock interviews and personalized feedback can be integrated.

10. Q&AI: An AI Powered Mock Interview Bot for Enhancing the Performance of Aspiring Professionals (2024)

- Conference: IEEE International Conference on Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI).
- Algorithm Used: NLP, Resume-Based Question Generation, Sentiment and Emotion Analysis.
- Key Methodology: Interview questions are generated using candidate information, and interview performance is evaluated through AI-based analysis.
- Limitation: Resume-to-job-description matching is not fully integrated into the interview preparation process.
- Research Gap: A unified platform connecting resume analysis, personalized questions and improvement recommendations is needed.

11. Developing an Intelligent Integrated Software Agent for Streamlined Interview Automation and Enhanced Candidate Insights (2024)

- Conference: IEEE 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI).
- Algorithm Used: Conversational AI, Intelligent Agents, Speech and Emotion Analysis.
- Key Methodology: An intelligent software agent automates interview interactions and generates insights about candidates.
- Limitation: Automated evaluation may not capture the complete technical depth of candidate responses.
- Research Gap: Detailed answer evaluation and personalized technical feedback can improve interview preparation.

12. A Survey of AI-Driven Mock Interviews Using GenAI and Machine Learning (InterviewX) (2024)

- Conference: IEEE 4th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS).
- Algorithm Used: Generative AI, RAG, QLoRA and Machine Learning.
- Key Methodology: The study reviews AI-driven mock interview approaches for generating domain-specific questions and providing feedback.
- Limitation: The study primarily reviews existing approaches rather than implementing a complete integrated system.
- Research Gap: Practical integration of resume analysis, adaptive interviews and personalized feedback is required.

13. AI Based Mock Interview System Using Natural Language Processing (2025)

- Conference: IEEE International Conference on Advanced Computing Technologies (ICoACT).
- Algorithm Used: NLP, Deep Learning, Speech-to-Text, Text-to-Speech, Reinforcement Learning and Computer Vision.

- Key Methodology: AI and NLP techniques are used to simulate interview interactions and support automated interview practice.
- Limitation: Multiple processing modalities may increase implementation complexity.
- Research Gap: A lightweight adaptive interview system can provide accessible and personalized preparation.

14. An Intelligent System for Automated Interview Question Generation: A Comparative Analysis of LLM Adaptation Techniques (2025)

- Conference: IEEE Tenth International Conference on Science Technology Engineering and Mathematics (ICONSTEM).
- Algorithm Used: RAG, Direct Fine-tuning, Mixture of Experts and Freeze+LoRA.
- Key Methodology: Different LLM adaptation techniques are compared for generating relevant interview questions and evaluating output quality.
- Limitation: The primary focus is on interview question generation.
- Research Gap: Resume-based skill-gap analysis and adaptive answer evaluation can be integrated.

15. Development of Adaptive Interview Evaluation System using NLP-Powered LLMs and Retrieval-Augmented Generation (2026)

- Conference: IEEE International Conference on Cognitive Computing and Networking Systems (ICC-CNS).
- Algorithm Used: Local LLMs, RAG, FAISS, MiniLM Embeddings and Ollama.
- Key Methodology: Resume information and retrieval techniques are used with LLMs to generate personalized interview questions and evaluate candidate responses.
- Limitation: Evaluation quality depends on retrieval accuracy and LLM-generated feedback.
- Research Gap: Resume enhancement, adaptive follow-up questions and personalized learning plans can be further integrated.

4. PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

4.1 Overall Workflow

The proposed system follows an integrated workflow consisting of resume analysis, job description matching, skill-gap identification, resume improvement, personalized interview preparation, and performance evaluation.

Initially, the candidate uploads a resume in PDF format and enters the target job role along with the job description. The system extracts the resume text using PyMuPDF and processes the provided information. The extracted resume content and job description are converted into semantic embeddings using the Nomic Embedding model. Cosine similarity is used to estimate semantic relevance between the resume and job description.

The Llama 3.2 language model analyzes the extracted information to identify matching skills, missing requirements, relevant experience, and areas requiring improvement. Based on this analysis, the system generates personalized resume improvement suggestions and interview preparation questions. During the mock interview, the system generates questions based on the candidate's resume and target job role. It also generates follow-up questions based on the candidate's previous answers to simulate an interactive interview experience.

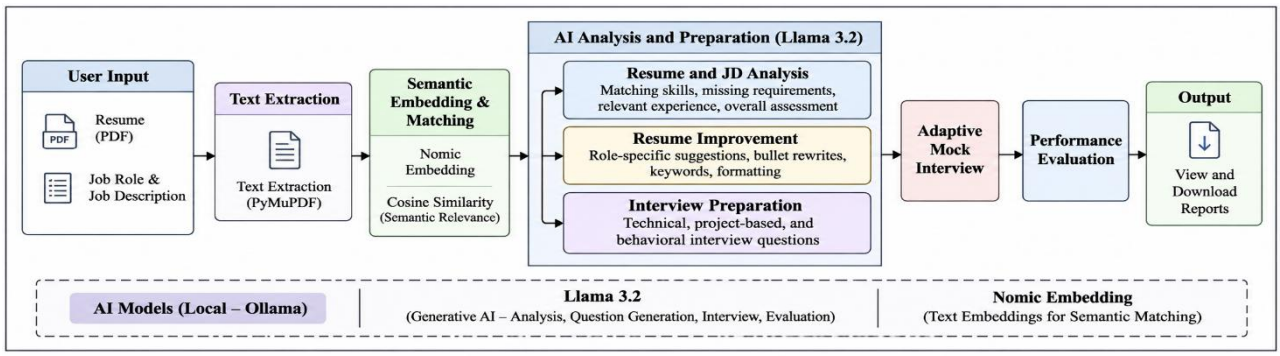


Figure 1: Proposed System Architecture of the AI Resume & Interview Assistant

Finally, the system evaluates the interview responses and generates a performance report containing scores, strengths, improvement areas, practice questions, and a personalized learning plan.

Figure 1. Proposed System Architecture

4.2 Main Components

Component	Purpose
Resume Upload	Accepts the candidate's resume in PDF format.
Text Extraction	Extracts textual information from the uploaded resume using PyMuPDF.
Job Description Input	Collects the target job role and job description.
Embedding Model	Generates semantic embeddings using Nomic Embedding.
Resume Matcher	Calculates cosine similarity between resume and job description embeddings.
Resume Analyzer	Identifies matching skills, missing requirements, and relevant experience using Llama 3.2.
Resume Improvement Module	Generates suggestions to improve resume content and highlight relevant skills.
Interview Preparation Module	Generates role-specific technical, project-based, and behavioral questions.
Adaptive Interview Module	Generates interview questions and follow-up questions based on candidate responses.
Evaluation Module	Evaluates interview responses and identifies strengths and improvement areas.
User Interface	Displays analysis results, interview interactions, and downloadable reports using Streamlit.

4.3 PROCESSING AND GENERATION PROCEDURE

The AI Resume & Interview Assistant follows a systematic processing and generation procedure to provide personalized career preparation. The system accepts the candidate's resume and target job description as input, processes the information using natural language processing and semantic embedding techniques, and utilizes a locally hosted Large Language Model (Llama 3.2) to generate resume analysis, improvement recommendations and interview preparation materials. The complete workflow consists of the following steps.

Step 1: Resume Upload

The process begins when the candidate uploads their resume in PDF format through the application interface. The resume contains important information about the candidate, including educational qualifications, technical skills, projects, certifications, internships and work experience. The uploaded document acts as the primary source of information for further analysis.

The application accepts the PDF file through the Streamlit interface. Once the file is uploaded, it is made available to the text extraction module for further processing. This step allows candidates to use their existing resumes without manually entering all their personal and professional details into the system.

Step 2: Text Extraction

After the resume is uploaded, the system extracts its textual content using the PyMuPDF library. PyMuPDF is a Python-based library that enables the application to read PDF documents and retrieve their text.

The extracted text is collected and prepared for subsequent processing. Information such as candidate details, educational qualifications, technical skills, project descriptions, certifications and professional experience can then be examined by the system.

This step is important because the original PDF document cannot be directly used for semantic comparison in its document format. Converting it into readable text makes the information accessible to the NLP and language model components. The extracted content is therefore passed to the next stage for comparison with the target job description.

Step 3: Job Description Input

After resume extraction, the candidate enters the target job role and its corresponding job description through the application interface. The job description generally contains the responsibilities, technical skills, educational requirements, experience and qualifications expected by the employer.

For example, a candidate applying for a Machine Learning Engineer position may provide a job description mentioning Python, machine learning algorithms, deep learning, data preprocessing and model deployment.

The system uses this information as a reference for evaluating the candidate's resume. Providing a specific job role and description helps the application generate more relevant recommendations instead of producing general resume suggestions. Both the extracted resume text and the job description are forwarded to the semantic matching module.

Step 4: Semantic Embedding and Matching

In this stage, the extracted resume text and job description are converted into numerical vector representations using the Nomic Embedding model. Text embeddings represent the semantic meaning of textual information in a numerical format, allowing the system to compare two different pieces of text based on their contextual relevance.

The application uses cosine similarity to estimate the semantic similarity between the resume and the job description. This technique measures the similarity between their vector representations. A higher similarity value indicates that the two text representations are more closely related in the embedding space.

Unlike simple keyword matching, semantic similarity can identify relationships between terms that have similar meanings even when their exact wording differs. For example, related descriptions of machine learning experience may be recognized as relevant even if the same keywords are not used.

The matching result provides an additional indication of how closely the candidate's profile relates to the selected job requirements. This information supports the subsequent resume analysis process.

Step 5: Resume and Skill Analysis

Once semantic matching is completed, the system uses the locally hosted Llama 3.2 model through Ollama to analyze the extracted resume content along with the provided job description.

The model examines the candidate's qualifications, technical skills, projects and experience in relation to the requirements of the selected job role. It identifies skills and qualifications that match the job description, requirements that appear to be missing, and areas where the candidate's resume may need improvement.

For example, if a job description requires Python, SQL, Machine Learning and Deep Learning, the system examines whether these skills are mentioned or supported by relevant project experience in the resume.

The analysis helps candidates understand which parts of their profile are relevant to the target position and which requirements may need additional attention. The results are generated as readable explanations rather than only presenting a numerical similarity score.

Step 6: Resume Improvement

Based on the analysis results, the Llama 3.2 model generates personalized resume improvement recommendations. The purpose of this stage is to help candidates present their existing qualifications, skills and experience more effectively.

The system suggests improvements to resume content, including clearer descriptions of academic projects, better presentation of technical skills, relevant keywords and improvements to bullet points. It may also recommend improving the organization and formatting of resume sections.

For instance, if a project description is too general, the system may suggest rewriting it to highlight the technologies used, the candidate's contribution and the outcome of the project.

The recommendations are based on the information provided by the candidate and the requirements of the target job role. The system is intended to improve the presentation of existing qualifications and should not invent skills, certifications, achievements or professional experience that the candidate does not possess.

Step 7: Interview Preparation

After resume analysis and improvement suggestions, the system generates interview preparation questions based on the candidate's resume and selected job role.

The Llama 3.2 model generates different categories of questions, including:

- **Technical Questions:** Questions related to programming languages, technical skills, tools, frameworks and concepts mentioned in the resume or job description.
- **Project-Based Questions:** Questions about the candidate's academic projects, implementation methods, technologies used, challenges faced and individual contributions.
- **Behavioral Questions:** Questions designed to understand communication, teamwork, problem-solving abilities, decision-making and professional attitude.

The generated questions are intended to reflect the candidate's profile rather than relying entirely on a fixed set of generic interview questions. This helps candidates prepare for questions that may be relevant to their actual skills and project experiences.

Step 8: Adaptive Mock Interview

The application provides an interactive mock interview environment in which candidates can answer the questions generated by the system. The candidate submits responses through the application interface, and the Llama 3.2 model processes the answers to continue the conversation.

The system supports adaptive questioning by generating relevant follow-up questions based on the candidate's previous responses. For example, if a candidate explains that they used a particular machine learning algorithm in an academic project, the model may ask why that algorithm was selected, how it was implemented or what challenges occurred during development.

This approach creates a conversational interview experience instead of simply displaying a fixed list of questions. It encourages candidates to explain their understanding in greater detail and practise responding to questions related to their own projects and technical knowledge.

Step 9: Performance Evaluation

After the mock interview session, the system analyzes the candidate's responses and generates an interview performance evaluation. The Llama 3.2 model examines the interview transcript to identify the quality and relevance of the answers.

The evaluation provides information about the candidate's strengths, areas requiring improvement, response quality and understanding of the questions. Performance scores and feedback are presented to help candidates recognize how effectively they answered the interview questions.

The system also identifies topics that may require further preparation and provides suggestions for improving future responses. This evaluation is intended as AI-generated practice feedback and should not be considered an official or guaranteed measure of interview performance.

Step 10: Feedback and Output Generation

In the final stage, the application presents the generated results to the candidate in an organized and accessible format. The output includes resume analysis, skill-matching information, resume improvement recommendations, interview questions and mock interview evaluation.

The candidate can review the feedback and access the available downloadable reports through the application interface. The generated information can be used to identify skill gaps, improve resume content and prepare for future technical and behavioral interviews.

The system also provides personalized learning recommendations based on the areas identified for improvement. These recommendations help candidates focus on relevant technical topics and strengthen their interview preparation.

Overall, the processing and generation procedure combines PDF text extraction, semantic embedding, cosine similarity and locally hosted generative AI to create an integrated resume analysis and interview preparation platform. By connecting resume improvement with personalized interview practice and performance feedback, the system aims to support candidates throughout their career preparation process.

5. TOOLS, TECHNOLOGIES AND KNOWLEDGE SOURCES

Layer	Technology Used
Programming Language	Python
Large Language Model	Llama 3.2
AI Model Runtime	Ollama
Embedding Model	Nomic Embedding (nomic-embed-text)
Semantic Similarity	Cosine Similarity
Application Framework	Streamlit
PDF Processing	PyMuPDF
API Communication	Python Requests
Development Environment	Visual Studio Code

5.1 Knowledge Sources / Dataset

The system does not use a separate model-training dataset. It uses user-provided PDF resumes and job descriptions as input sources for analysis. Candidate interview responses are also used to generate personalized feedback and performance evaluation. The system processes this information using locally running AI models through Ollama.

5.2 Reproducibility Details

The application is developed using Python and Streamlit. Llama 3.2 and Nomic Embedding are executed locally through Ollama. PyMuPDF is used for PDF text extraction, while the Requests library communicates with the Ollama API. Reproducing the system requires Python, the required Python packages, Ollama, and the specified models to be installed and configured.

6. IMPLEMENTATION

The AI Resume & Interview Assistant is implemented using Python and Streamlit, integrating locally hosted Artificial Intelligence models through Ollama. The application combines document processing, semantic similarity analysis and Large Language Model (LLM) capabilities to provide personalized resume analysis and interview preparation. The implementation is organized into several modules, each performing a specific function in the overall workflow.

6.1 Data Preparation

The application accepts resumes in PDF format along with the target job role and job description provided by the user. PyMuPDF is used to extract textual information from uploaded resumes, including educational qualifications, technical skills, projects, certifications and professional experience. The extracted text is checked and prepared for further processing. This converts the resume content into a readable format that can be used by the embedding model and LLM.

6.2 Embedding and Similarity Analysis

The Nomic Embedding model converts the extracted resume content and job description into numerical vector representations called embeddings. These embeddings capture the semantic meaning of the provided text and enable the system to compare the information based on contextual relevance. Cosine similarity is then applied to estimate the similarity between the resume and job description. The resulting similarity value provides an indication of how closely the candidate's profile matches the requirements of the selected job role.

6.3 Resume and Job Description Analysis

The Llama 3.2 model analyzes the extracted resume information along with the provided job description. It identifies skills that match the job requirements, missing or unclear qualifications and relevant academic or professional experience. Based on this analysis, the system provides an overall understanding of the candidate's profile in relation to the target position. The results are presented in a structured format to help candidates understand their strengths and identify areas that require improvement.

6.4 LLM Integration

Ollama is used to run the Llama 3.2 model locally, enabling the application to perform different generative AI operations. The model generates personalized resume improvement suggestions, including better descriptions of existing projects, relevant keywords and content improvements. It also generates technical, project-based and behavioral interview questions. During the mock interview, the model processes candidate responses and generates relevant follow-up questions. After the interview, it analyzes the responses and produces performance feedback with strengths and areas for improvement.

6.5 User Interface

The application interface is developed using Streamlit, which provides an interactive and user-friendly environment. It allows candidates to upload PDF resumes, enter job descriptions and access the different application functionalities. The interface displays resume analysis results, semantic matching information and improvement recommendations. It also provides interview preparation features, an interactive mock interview chatbot and a performance evaluation section. The generated reports can be viewed and downloaded through the application.

6.6 Error Handling

Error handling is incorporated to improve application reliability and ensure smooth operation. The application checks for invalid PDF uploads, empty resume content and missing job descriptions before proceeding with further processing. It also handles Ollama API connection failures, unexpected model responses and errors during AI-generated operations. Appropriate error messages are displayed to inform users about the issue and help them take corrective action.

Overall, the implementation integrates Python, PyMuPDF, Nomic Embedding, cosine similarity, Ollama, Llama 3.2 and Streamlit to develop an integrated AI-based resume analysis and interview preparation system. Each module contributes to the complete workflow, from resume upload and job matching to personalized interview practice and performance evaluation.

7. EXPERIMENTS, RESULTS AND DISCUSSION

7.1 Evaluation Design

The system is evaluated by testing its major functionalities using different resumes and job descriptions. The evaluation focuses on resume analysis, semantic similarity, skill matching, resume improvement, interview question generation, adaptive interview performance, and feedback generation.

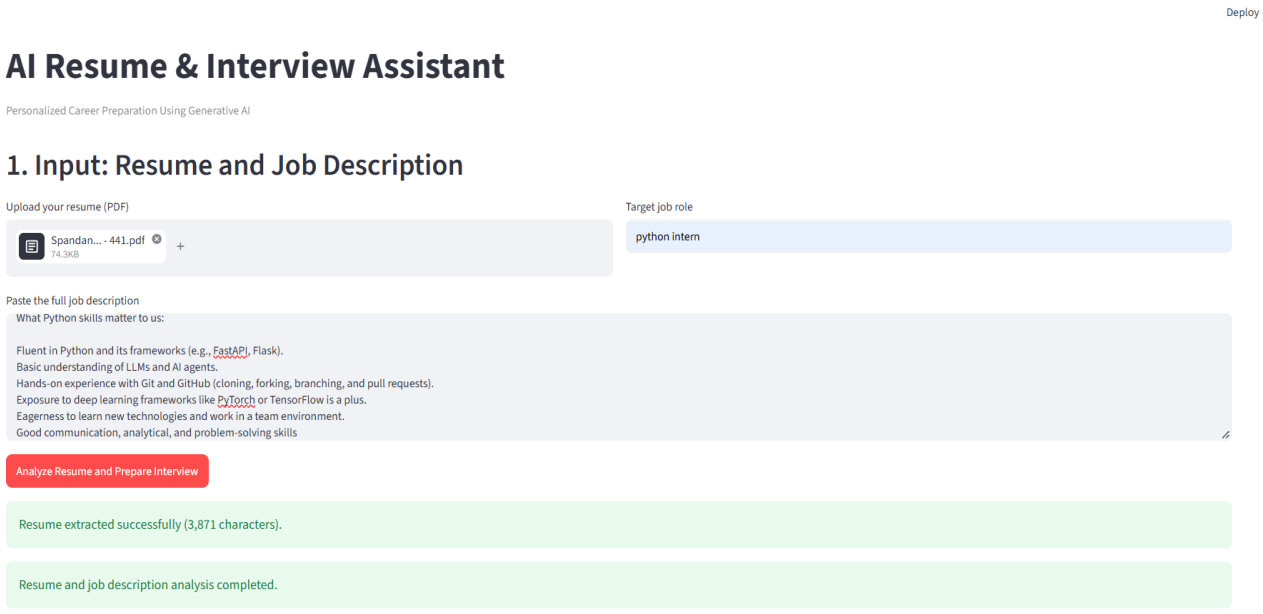
Evaluation Metrics

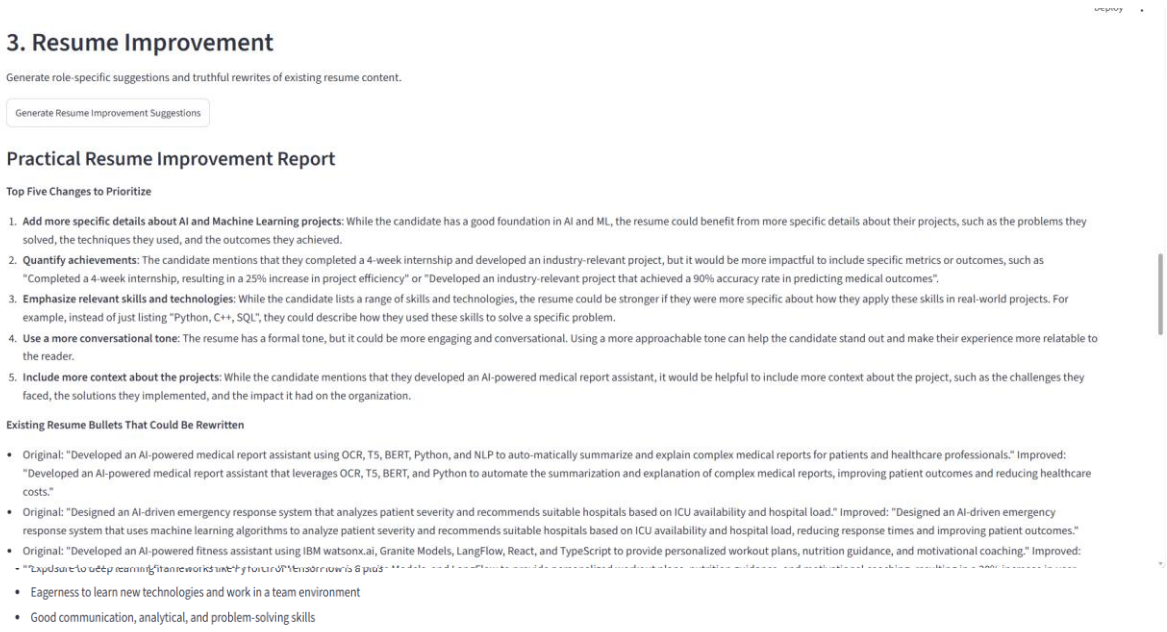
Metric	What It Measures
Semantic Similarity	Measures the relevance between resume content and job description using cosine similarity.
Skill Matching	Checks whether relevant job requirements are identified in the resume.
Resume Improvement Quality	Evaluates the relevance and usefulness of generated resume suggestions.
Question Relevance	Measures how well generated interview questions match the candidate's profile and target role.
Adaptive Interview Performance	Checks whether follow-up questions consider previous candidate responses.
Evaluation Quality	Assesses the relevance and usefulness of generated interview feedback.
Response Time	Measures the time taken by the application to complete AI operations.
User Feedback	Collects feedback on usability and the usefulness of the generated results.

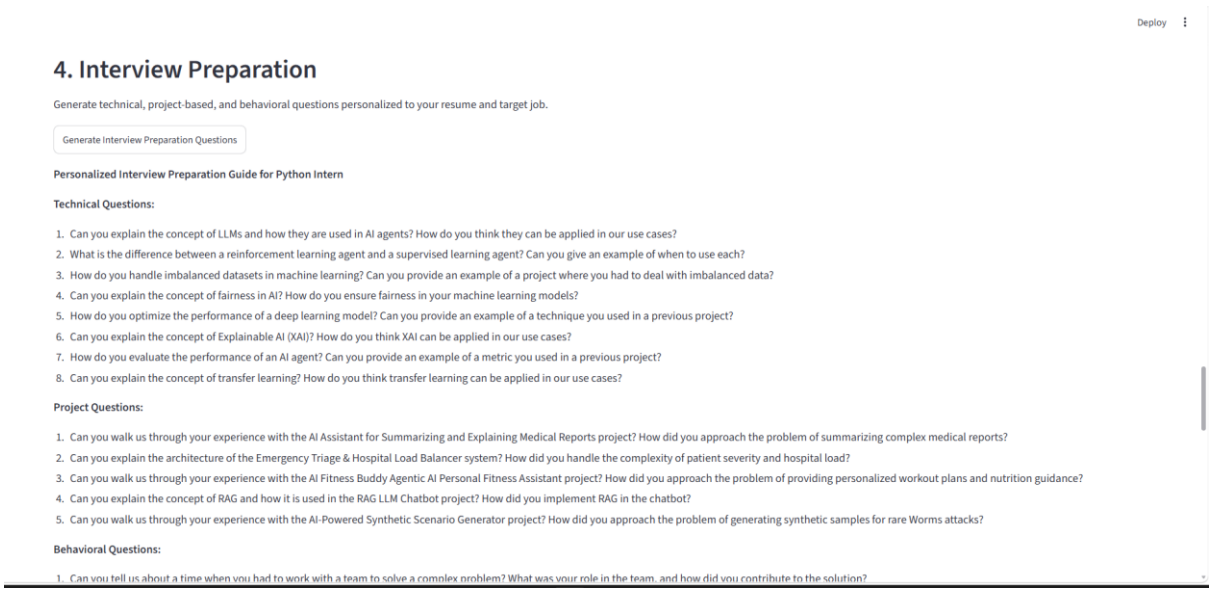
7.2 Results and Discussion

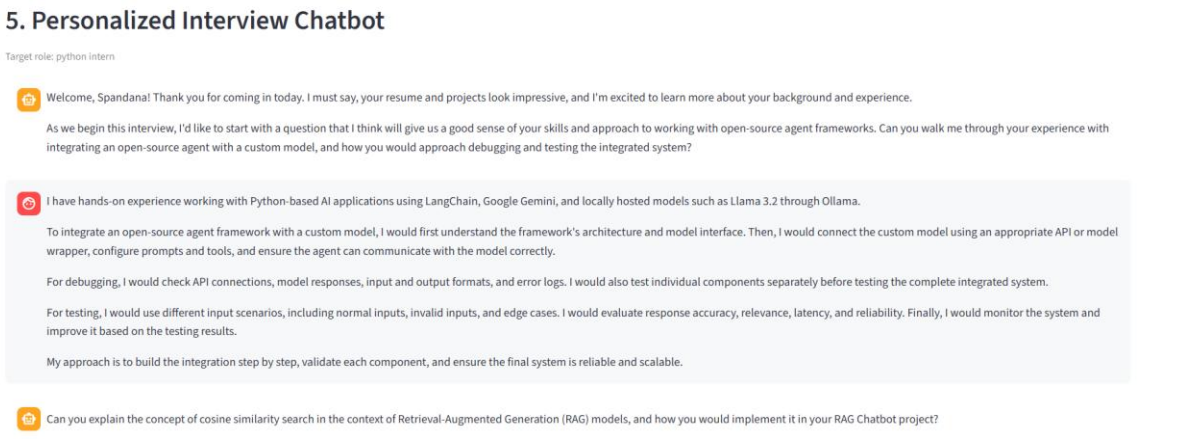
The application successfully demonstrates resume text extraction, resume and job description analysis, personalized resume improvement suggestions, interview question generation, adaptive interview conversations, and AI-generated evaluation reports.

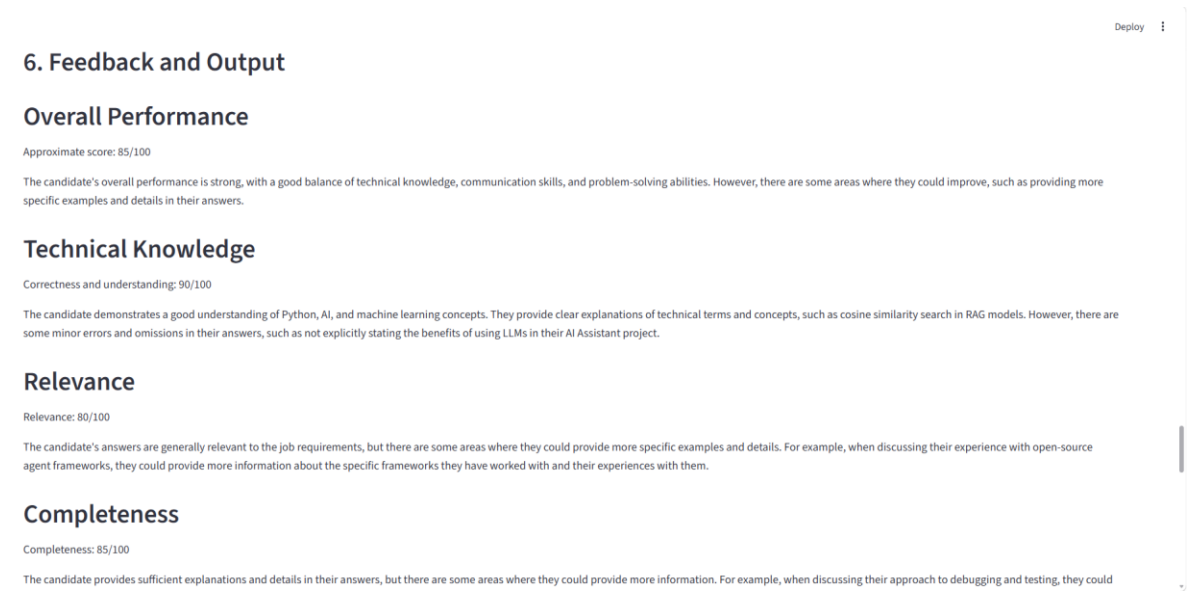
The results demonstrate how Generative AI can support personalized career preparation through an integrated application. However, semantic similarity and AI-generated evaluation scores are approximate and require further validation using a larger test set and user feedback.






Deploy
4. Interview Preparation
Generate technical, project-based, and behavioral questions personalized to your resume and target job.
Generate Interview Preparation Questions
Personalized Interview Preparation Guide for Python Intern
Technical Questions:
1. Can you explain the concept of LLMs and how they are used in AI agents? How do you think they can be applied in our use cases?
2. What is the difference between a reinforcement learning agent and a supervised learning agent? Can you give an example of when to use each?
3. How do you handle imbalanced datasets in machine learning? Can you provide an example of a project where you had to deal with imbalanced data?
4. Can you explain the concept of fairness in AI? How do you ensure fairness in your machine learning models?
5. How do you optimize the performance of a deep learning model? Can you provide an example of a technique you used in a previous project?
6. Can you explain the concept of Explainable AI (XAI)? How do you think XAI can be applied in our use cases?
7. How do you evaluate the performance of an AI agent? Can you provide an example of a metric you used in a previous project?
8. Can you explain the concept of transfer learning? How do you think transfer learning can be applied in our use cases?
Project Questions:
1. Can you walk us through your experience with the AI Assistant for Summarizing and Explaining Medical Reports project? How did you approach the problem of summarizing complex medical reports?
2. Can you explain the architecture of the Emergency Triage & Hospital Load Balancer system? How did you handle the complexity of patient severity and hospital load?
3. Can you walk us through your experience with the AI Fitness Buddy Agentic AI Personal Fitness Assistant project? How did you approach the problem of providing personalized workout plans and nutrition guidance?
4. Can you explain the concept of RAG and how it is used in the RAG LLM Chatbot project? How did you implement RAG in the chatbot?
5. Can you walk us through your experience with the AI-Powered Synthetic Scenario Generator project? How did you approach the problem of generating synthetic samples for rare Worms attacks?
Behavioral Questions:
1. Can you tell us about a time when you had to work with a team to solve a complex problem? What was your role in the team, and how did you contribute to the solution?


5. Personalized Interview Chatbot
Target role: python intern
Welcome, Spandana! Thank you for coming in today. I must say, your resume and projects look impressive, and I'm excited to learn more about your background and experience.
As we begin this interview, I'd like to start with a question that I think will give us a good sense of your skills and approach to working with open-source agent frameworks. Can you walk me through your experience with integrating an open-source agent with a custom model, and how you would approach debugging and testing the integrated system?
I have hands-on experience working with Python-based AI applications using LangChain, Google Gemini, and locally hosted models such as Llama 3.2 through Ollama.
To integrate an open-source agent framework with a custom model, I would first understand the framework's architecture and model interface. Then, I would connect the custom model using an appropriate API or model wrapper, configure prompts and tools, and ensure the agent can communicate with the model correctly.
For debugging, I would check API connections, model responses, input and output formats, and error logs. I would also test individual components separately before testing the complete integrated system.
For testing, I would use different input scenarios, including normal inputs, invalid inputs, and edge cases. I would evaluate response accuracy, relevance, latency, and reliability. Finally, I would monitor the system and improve it based on the testing results.
My approach is to build the integration step by step, validate each component, and ensure the final system is reliable and scalable.
Can you explain the concept of cosine similarity search in the context of Retrieval-Augmented Generation (RAG) models, and how you would implement it in your RAG Chatbot project?


Deploy
6. Feedback and Output
Overall Performance
Approximate score: 85/100
The candidate's overall performance is strong, with a good balance of technical knowledge, communication skills, and problem-solving abilities. However, there are some areas where they could improve, such as providing more specific examples and details in their answers.
Technical Knowledge
Correctness and understanding: 90/100
The candidate demonstrates a good understanding of Python, AI, and machine learning concepts. They provide clear explanations of technical terms and concepts, such as cosine similarity search in RAG models. However, there are some minor errors and omissions in their answers, such as not explicitly stating the benefits of using LLMs in their AI Assistant project.
Relevance
Relevance: 80/100
The candidate's answers are generally relevant to the job requirements, but there are some areas where they could provide more specific examples and details. For example, when discussing their experience with open-source agent frameworks, they could provide more information about the specific frameworks they have worked with and their experiences with them.
Completeness
Completeness: 85/100
The candidate provides sufficient explanations and details in their answers, but there are some areas where they could provide more information. For example, when discussing their approach to debugging and testing, they could

8. Limitations, Ethical Considerations and Risk Mitigation

Although the proposed Retrieval-Augmented Generation (RAG) system provides context-aware responses based on retrieved documents, it has certain limitations that may affect its performance and reliability.

8.1 Limitations

- The system may generate incorrect, incomplete, or irrelevant responses if the retrieved information is insufficient.
- Poor-quality, scanned, or incorrectly formatted PDF documents may affect text extraction and retrieval accuracy.
- The retrieval process may fail to identify relevant information when documents contain complex terminology or ambiguous content.
- The language model may occasionally generate unsupported information (hallucinations) despite being provided with retrieved context.
- The system's performance depends on the quality, completeness, and availability of the uploaded documents.

8.2 Ethical Considerations

The system must be developed and used responsibly to ensure data privacy, transparency, and reliability.

- Only authorized documents should be uploaded and processed.
- Source references should be displayed wherever possible to support answer verification.
- The system should clearly communicate when information is unavailable in the indexed documents.
- Confidential academic, personal, or institutional information must be protected.
- External language model APIs should be used with proper consideration of data privacy and retention policies.
- The system should be tested against prompt injection attacks and malicious instructions embedded in uploaded documents.

8.3 Risk Mitigation Strategies

To minimize potential risks, the following measures are considered:

- Validate uploaded documents before processing.
- Apply appropriate text cleaning and preprocessing techniques.
- Use reliable embedding models and similarity-based retrieval techniques.
- Restrict answer generation to the retrieved context wherever possible.
- Implement error handling for missing documents, API failures, and invalid user inputs.
- Evaluate the system using different test queries to measure retrieval quality and response accuracy.

9. Conclusion and Future Enhancements

9.1 Conclusion

The AI Resume & Interview Assistant demonstrates the application of Generative Artificial Intelligence and Natural Language Processing in improving the resume preparation and interview preparation process. The proposed system provides an integrated platform that helps candidates analyze their resumes, understand job requirements, identify skill gaps and practise interviews based on their individual profiles.

The system combines several technologies, including Python, Streamlit, PyMuPDF, Nomic Embedding, cosine similarity, Ollama and the Llama 3.2 Large Language Model. The process begins with uploading a candidate's resume in PDF format and providing the target job role and corresponding job description. PyMuPDF extracts the textual information from the resume, which is subsequently processed to understand the candidate's educational qualifications, technical skills, projects and experience.

The Nomic Embedding model converts the resume content and job description into numerical vector representations. Cosine similarity is then used to estimate their semantic relevance. This approach helps identify the relationship between the candidate's profile and the requirements of the selected job role. The system further utilizes Llama 3.2, running locally through Ollama, to analyze matching skills, identify missing or unclear requirements and provide personalized resume improvement recommendations.

Another important feature of the proposed system is its AI-powered interview preparation module. Based on the candidate's resume and target job role, the application generates technical, project-based and behavioral interview questions. The adaptive mock interview functionality allows candidates to interact with the AI assistant and receive follow-up questions based on their previous responses. This provides an opportunity to practise answering interview questions in a conversational environment rather than depending entirely on predefined question lists.

The performance evaluation module further analyzes the candidate's interview responses and generates feedback highlighting strengths, weaknesses and areas requiring improvement. The application also provides recommendations that can help candidates improve their technical knowledge and interview performance. The generated results and reports make it easier for

candidates to review their preparation progress and identify topics that require additional attention.

The project demonstrates how locally hosted Generative AI models can be integrated with semantic text processing techniques to develop a practical career preparation application. By combining resume analysis, job-description matching, resume improvement, personalized question generation and mock interview evaluation within a single platform, the system aims to make interview preparation more structured, accessible and personalized.

Although the proposed application provides several useful functionalities, its performance depends on the quality of the uploaded resume, the completeness of the job description, the semantic matching process and the capabilities of the selected language model. AI-generated feedback may also require manual verification to ensure that the recommendations are accurate and relevant. Overall, the project provides a practical demonstration of how Generative AI can support students and job seekers in preparing for employment opportunities and improving their professional presentation.

9.2 Future Enhancements

Although the current system provides essential resume analysis and interview preparation functionalities, several improvements can be introduced in future versions to enhance its accuracy, usability, accessibility and overall performance.

1. Multilingual Support: Future versions can incorporate multilingual capabilities to allow candidates to interact with the application in different languages. This would make the system more accessible to users who are more comfortable preparing for interviews in their preferred language.

2. Voice-Based Mock Interviews: The current interview interaction can be extended with speech-to-text and text-to-speech technologies. Candidates could answer questions verbally instead of typing their responses. This would provide a more realistic interview experience and help users practise communication and spoken response skills.

3. Advanced Resume Parsing: The system can be enhanced with Optical Character Recognition (OCR) to extract information from scanned PDF documents and image-based resumes. This would improve compatibility with different resume formats and document types.

4. Improved Semantic Matching: Future implementations can explore hybrid matching techniques that combine keyword-based matching with semantic embeddings. This may improve the identification of relevant skills, technical terms and qualifications in resumes, even when different terminology is used.

5. Conversation Memory and Personalized Follow-ups: Conversation memory can be introduced to maintain the context of previous interview interactions. This would enable the assistant to generate more relevant follow-up questions based on earlier responses and provide a continuous interview experience.

6. Advanced Interview Evaluation: The evaluation module can be improved by introducing more detailed assessment criteria, including technical correctness, relevance, completeness, clarity and problem-solving ability. Additional evaluation methods and human-reviewed test cases can help assess the reliability of AI-generated feedback.

7. Personalized Learning Recommendations: The application can be extended to generate structured learning plans based on the candidate's identified skill gaps. It could recommend relevant topics, programming exercises, technical concepts and preparation activities to help candidates strengthen their knowledge before attending interviews.

8. Resume Version Comparison: A future enhancement could allow candidates to upload multiple versions of their resumes and compare them against the same job description. This would help users understand whether the changes made to their resumes improve their relevance to the target role.

9. Cloud Deployment and Scalability: The application can be deployed on a cloud platform to make it accessible to a larger number of users. Performance optimization, efficient model management and suitable application deployment architecture can help support multiple users and larger workloads.

10. Privacy and Security Improvements: Since resumes contain personal and professional information, future versions can introduce secure authentication, access control and encrypted data handling. These improvements would help protect candidate information and provide a more secure application environment.

In conclusion, the proposed AI Resume & Interview Assistant provides a foundation for developing more advanced AI-powered career preparation tools. Incorporating these future

enhancements could make the application more interactive, intelligent, scalable and useful for students and job seekers preparing for competitive employment opportunities.

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[4] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.